COS214 Practical 3

Ethan Van Eyden (u25260244)

Mogale Kagiso Lebethe (u25237269)

Lesego Tebeile (u25143230)

Table of Contents

Task 1: Event description

The **Tech Expo** is an event where visitors explore different areas of technology, attend keynote presentations and demonstrations, and interact with specialised technology exhibits. The expo is organised as a hierarchy of areas and event units using the **Composite pattern**, allowing both individual units and groups of units to be managed consistently.

ExhibitionHall is the root of the expo and contains the major areas, including MainStage and AIZone. The MainStage provides areas for presentations and demonstrations, containing DemoArea and KeyNoteArea. AIZone focuses on artificial intelligence exhibits and contains AIDemoBooth and RoboticsBooth. ByteSizedBites provides food and refreshments for visitors. These areas and units form the main components of the Composite tree.

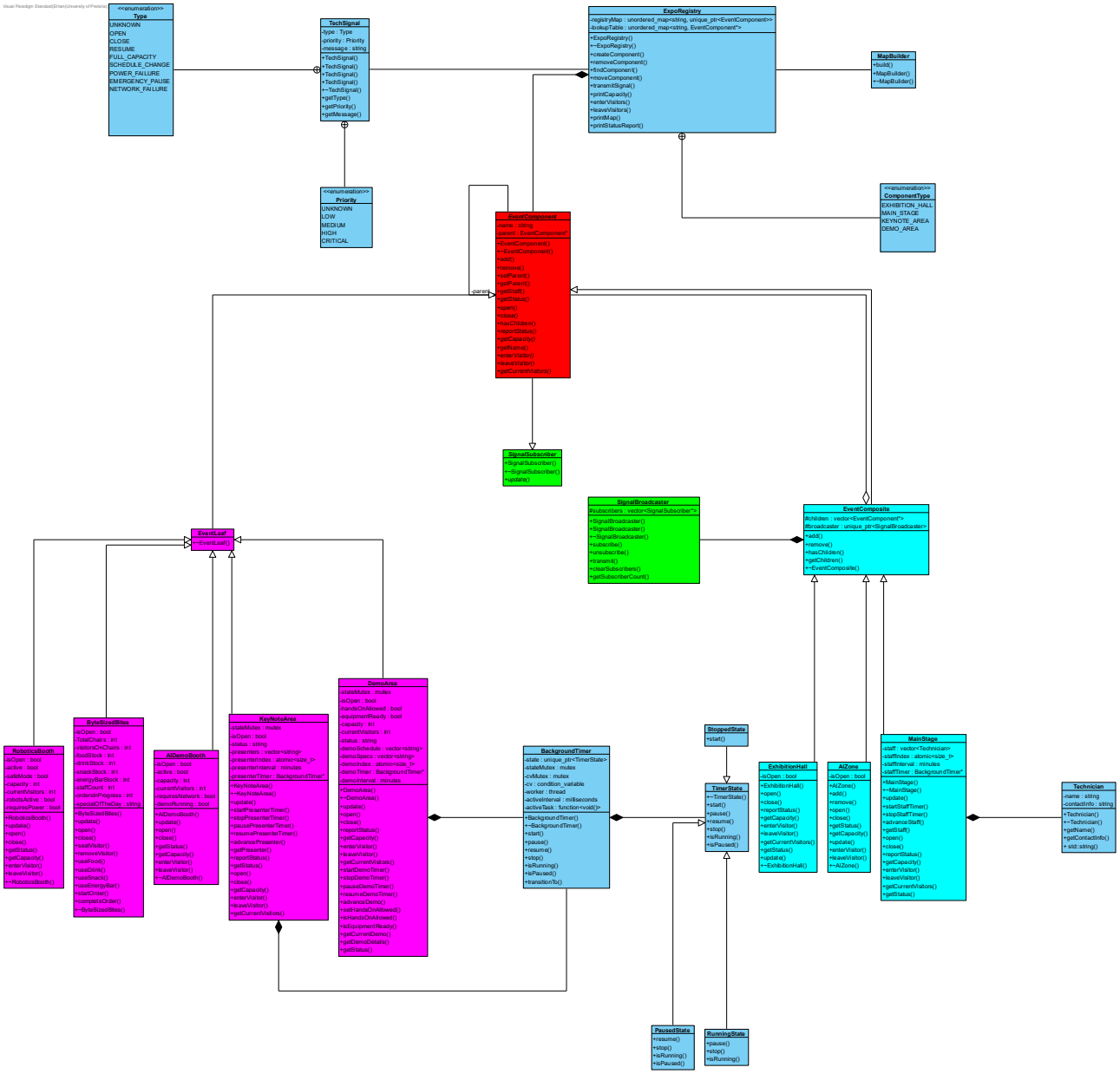
During the expo, areas can be opened and closed, while visitor capacity and current attendance are tracked throughout the hierarchy. Visitor entry is managed according to the available capacity of each area. Technicians are assigned to areas and can rotate during the event, ensuring that technical support is available where required.

The expo also handles technical signals, which can be propagated through the hierarchy so that relevant components can respond to events such as technical issues or notifications. Each composite coordinates its child components, while leaf units represent individual facilities that do not contain further components. The world builder constructs the complete expo by establishing the relationships between the ExhibitionHall, its composite areas, and leaf event units.

Task 2: Participants

- Observer: SignalSubscriber
- Subject: SignalBroadcaster
- EventComponent: Component and ConcreteObserver
- EventLeaf: Leaf (abstract interface) and ConcreteObserver. Leaves are the bottom of the composite tree hierarchy. They should be able to receive signals from up the tree and process them, but since they have no children, they have no need to transmit it further. It inherits both from the EvenComponent and indirectly from SignalSubscriber.
- EventComposite: Composite (abstract interface), ConcreteObserver and ConcreteSubject. Composite nodes are intermediate nodes on the tree hierarchy. They should be able to receive messages, process them and transmit it down the tree to all of its children. It inherits from EventComponent and indirectly from SignalSubscriber. It is however a ConcreteSubject via composition to allow flexibility (e.g. the nodes broadcaster is broken, and its children receive no messages or to potentially allow different broadcast strategies).
- MainStage: Composite, ConcreteObserver and ConcreteSubject
- ExibitionHall: Composite, ConcreteObserver and ConcreteSubject
- AIZone: Composite, ConcreteObserver and ConcreteSubject
- KeyNoteArea: Leaf and ConcreteObserver
- DemoArea: Leaf and ConcreteObserver
- AiDemoBooth: Leaf and ConcreteObserver
- RoboticsBooth: Leaf and ConcreteObserver
- ByteSizedBites: Leaf and ConcreteObserver

Task 3: UML


<<enumeration>>
Type
UNKNOWN
OPEN
CLOSE
RESUME
FULL_CAPACITY
SCHEDULE_CHANGE
POWER_FAILURE
EMERGENCY_PAUSE
NETWORK_FAILURE
TechSignal
-type : Type
-priority : Priority
-message : string
+TechSignal()
+TechSignal()
+TechSignal()
+TechSignal()
+~TechSignal()
+getType()
+getPriority()
+getMessage()
<<enumeration>>
Priority
UNKNOWN
LOW
MEDIUM
HIGH
CRITICAL
ExpoRegistry
-registryMap : unordered_map<string, unique_ptr<EventComponent>>
-lookupTable : unordered_map<string, EventComponent*>
+ExpoRegistry()
+~ExpoRegistry()
+createComponent()
+removeComponent()
+findComponent()
+moveComponent()
+transmitSignal()
+printCapacity()
+enterVisitors()
+leaveVisitors()
+printMap()
+printStatusReport()
MapBuilder
+build()
+MapBuilder()
+~MapBuilder()
<<enumeration>>
ComponentType
EXHIBITION_HALL
MAIN_STAGE
KEYNOTE_AREA
DEMO_AREA
EventComponent
-name : string
-parent : EventComponent*
+EventComponent()
+~EventComponent()
+add()
+remove()
+setParent()
+getParent()
+getStaff()
+getStatus()
+open()
+close()
+hasChildren()
+reportStatus()
+getCapacity()
+getName()
+enterVisitor()
+leaveVisitor()
-getCurrentVisitors()
parent
SignalSubscriber
+SignalSubscriber()
+~SignalSubscriber()
+update()
SignalBroadcaster
#subscribers : vector<SignalSubscriber*>
+SignalBroadcaster()
+~SignalBroadcaster()
+subscribe()
+unsubscribe()
+transmit()
+clearSubscribers()
+getSubscriberCount()
EventComposite
#children : vector<EventComponent*>
#broadcaster : unique_ptr<SignalBroadcaster>
+add()
+remove()
+hasChildren()
+getChildren()
+~EventComposite()
EventLeaf
+~EventLeaf()
RoboticsBooth
-isOpen : bool
-active : bool
-safeMode : bool
-capacity : int
-currentVisitors : int
-robotsActive : bool
-requiresPower : bool
+RoboticsBooth()
+update()
+open()
+close()
+getStatus()
+getCapacity()
+enterVisitor()
+leaveVisitor()
+~RoboticsBooth()
ByteSizedBites
-isOpen : bool
-totalChairs : int
-visitorsOnChairs : int
-foodStock : int
-drinkStock : int
-snackStock : int
-energyBarStock : int
-staffCount : int
-ordersInProgress : int
-specialOfTheDay : string
+ByteSizedBites()
+update()
+open()
+close()
+seatVisitor()
+removeVisitor()
+useFood()
+useDrink()
+useSnack()
+useEnergyBar()
+startOrder()
+completeOrder()
+~ByteSizedBites()
AIDemoBooth
-isOpen : bool
-active : bool
-capacity : int
-currentVisitors : int
-requiresNetwork : bool
-demoRunning : bool
+AIDemoBooth()
+update()
+open()
+close()
+getStatus()
+getCapacity()
+enterVisitor()
+leaveVisitor()
+~AIDemoBooth()
KeyNoteArea
-stateMutex : mutex
-isOpen : bool
-status : string
-presenters : vector<string>
-presenterIndex : atomic<size_t>
-presenterInterval : minutes
-presenterTimer : BackgroundTimer*
+KeyNoteArea()
+~KeyNoteArea()
+update()
+startPresenterTimer()
+stopPresenterTimer()
+pausePresenterTimer()
+resumePresenterTimer()
+advancePresenter()
+getPresenter()
+reportStatus()
+getStatus()
+open()
+close()
+getCapacity()
+enterVisitor()
+leaveVisitor()
+getCurrentVisitors()
DemoArea
-stateMutex : mutex
-isOpen : bool
-handsOnAllowed : bool
-equipmentReady : bool
-capacity : int
-currentVisitors : int
-status : string
-demoSchedule : vector<string>
-demoSpecs : vector<string>
-demoIndex : atomic<size_t>
-demoTimer : BackgroundTimer*
-demoInterval : minutes
+DemoArea()
+~DemoArea()
+update()
+open()
+close()
+reportStatus()
+getCapacity()
+enterVisitor()
+leaveVisitor()
+getCurrentVisitors()
+startDemoTimer()
+stopDemoTimer()
+pauseDemoTimer()
+resumeDemoTimer()
+advanceDemo()
+setHandsOnAllowed()
+isHandsOnAllowed()
+isEquipmentReady()
+getCurrentDemo()
+getDemoDetails()
+getStatus()
BackgroundTimer
-state : unique_ptr<TimerState>
-stateMutex : mutex
-cvMutex : mutex
-cv : condition_variable
-worker : thread
-activeInterval : milliseconds
-activeTask : function<void()>
+BackgroundTimer()
+~BackgroundTimer()
+start()
+pause()
+resume()
+stop()
+isRunning()
+isPaused()
+transitionTo()
StoppedState
+start()
TimerState
+~TimerState()
+start()
+pause()
+resume()
+stop()
+isRunning()
+isPaused()
PausedState
+resume()
+stop()
+isRunning()
+isPaused()
RunningState
+pause()
+stop()
+isRunning()
ExhibitionHall
-isOpen : bool
+ExhibitionHall()
+open()
+close()
+reportStatus()
+getCapacity()
+enterVisitor()
+leaveVisitor()
+getCurrentVisitors()
+getStatus()
+update()
+~ExhibitionHall()
AIZone
-isOpen : bool
+AIZone()
+add()
+remove()
+open()
+close()
+getStatus()
+getCapacity()
+update()
+enterVisitor()
+leaveVisitor()
+~AIZone()
MainStage
-staff : vector<Technician>
-staffIndex : atomic<size_t>
-staffInterval : minutes
-staffTimer : BackgroundTimer*
+MainStage()
+~MainStage()
+update()
+startStaffTimer()
+stopStaffTimer()
+advanceStaff()
+getStaff()
+open()
+close()
+reportStatus()
+getCapacity()
+enterVisitor()
+leaveVisitor()
+getCurrentVisitors()
+getStatus()
Technician
-name : string
-contactInfo : string
+Technician()
+~Technician()
+getName()
+getContactInfo()
+ std::string()

Task 4: Design rationale

The hierarchy we made is a part-whole relationship. The tech expo map is divided into zones which form a part of the whole map. Each can be made up of other sub-zones or specific booths and stations. They all form part of the whole map but need to be treated uniformly.

The observer pattern is required as this application is event driven. A node must subscribe to its parent so that if there is an event that happens it is notified immediately. The main issue with Hard-coded calls is they tightly couple the parent to specific concrete children, whereas Observer lets the parent notify an arbitrary set of interested observers.

Event components are owned in a central class ExpoRegistry which is the API for the software. It has a lookup table to every component for command parsing. The composite and observer design patterns are just responsible for maintaining the structure of the hierarchy but don't own any memory. The reason for this is to simplify memory management significantly. We only need to keep track of object lifecycle from one place and that single map is the source of truth of whether a component exists or not. It uses unique pointer and thus will automatically fall out of scope if it is removed from the lookup. The ExpoRegistry does however still call methods to remove children of composites and unsubscribe observers before removing it from the lookup to avoid dangling pointers.

A class participating as both a Subject and an Observer does not violate the intent of either pattern. Intermediate Composite nodes need to receive notifications from components above them and, when appropriate, propagate those notifications to interested components below them. The Observer role describes the node's relationship with its parent, while the Subject role describes its relationship with its own observers. These are two separate collaborations serving different purposes, allowing notifications to cascade through the event hierarchy without introducing hard-coded dependencies between concrete event components.

Task 5: Object diagram

Task 6: Event rules and original features

4.1

Event rules The system uses the push form of Observer: each SignalBroadcaster passes the full TechSignal to update(const TechSignal&). The same notice therefore reaches all registered children, but each concrete leaf decides its own response from its state and responsibilities. No controller checks a concrete type.

1. Power failure: DemoArea closes the area, locks the equipment, disables hands-on use and pauses its rotating-demo timer. AIDemoBooth stops its AI demonstration. RoboticsBooth stops its showcase and enables safe mode. ByteSizedBites closes its food service. KeyNoteArea closes and pauses its presenter timer.

2. Emergency pause: DemoArea keeps its operational state but marks equipment not ready, disables hands-on use and pauses the timer. RoboticsBooth stops robots and enters safe mode. AIDemoBooth stops its demo. ByteSizedBites closes. KeyNoteArea pauses the presenter timer.

3. Resume: A previously open AIDemoBooth restarts its demonstration; a previously open RoboticsBooth restarts the robots and exits safe mode. ByteSizedBites reopens. KeyNoteArea resumes its presenter timer only when the area is open.

4. Network failure: AIDemoBooth pauses only when it requires network access. ByteSizedBites clears orders in progress because card/order processing is unavailable. Units that do not depend on the network remain unaffected.

5. Full capacity: AIDemoBooth and RoboticsBooth reject further admission and report that they are full. DemoArea records that its spectator zone is full.

ByteSizedBites marks all chairs occupied. A composite (AIZone, MainStage, or ExhibitionHall) distributes incoming visitors through children in order and reports any remainder that cannot be admitted.

6. Schedule change: KeyNoteArea, DemoArea, AIDemoBooth, and RoboticsBooth retain the notice in their status/output. The presentation and demonstration areas also use getStaff() to identify the technician available through the parent hierarchy.

4.2

Runtime reorganisation ExpoRegistry::moveComponent(childName, newParentName) finds both registered components and applies operator>>(child, newParent). The operator first validates that the move is not a self-move and cannot create a cycle. It then calls oldParent->remove(&child), which removes the non-owning child pointer, clears child.parent, and unsubscribes the child from the old parent broadcaster. Finally it calls newParent.add(&child), which records the new parent and subscribes the child to the new parent broadcaster.

ExpoRegistry remains the sole owner through registryMap<string, unique_ptr>; composites and broadcasters store non-owning raw pointers only. Moving Demo_Area from Main_Stage to Main_Exhibition_Hall therefore changes containment and notification registration but does not transfer or duplicate ownership. This avoids double deletion.

4.3

Condition-based decisions Capacity admission is the primary conditional behaviour. AIZone::enterVisitor(int) only admits visitors when the zone is open and the requested count is positive. It loops through its children, forwarding the remaining number of visitors. Each concrete booth admits min(requested, capacity - currentVisitors). When a booth reaches capacity it handles a FULL_CAPACITY signal; if visitors still remain after the loop,

the zone reports insufficient capacity. This decision appears as the alt and opt fragments in SD3.

DemoArea::setHandsOnAllowed(bool) supplies a second state-based rule: hands-on testing is enabled only if the area is open and its equipment is ready; otherwise it is forced off.

4.4

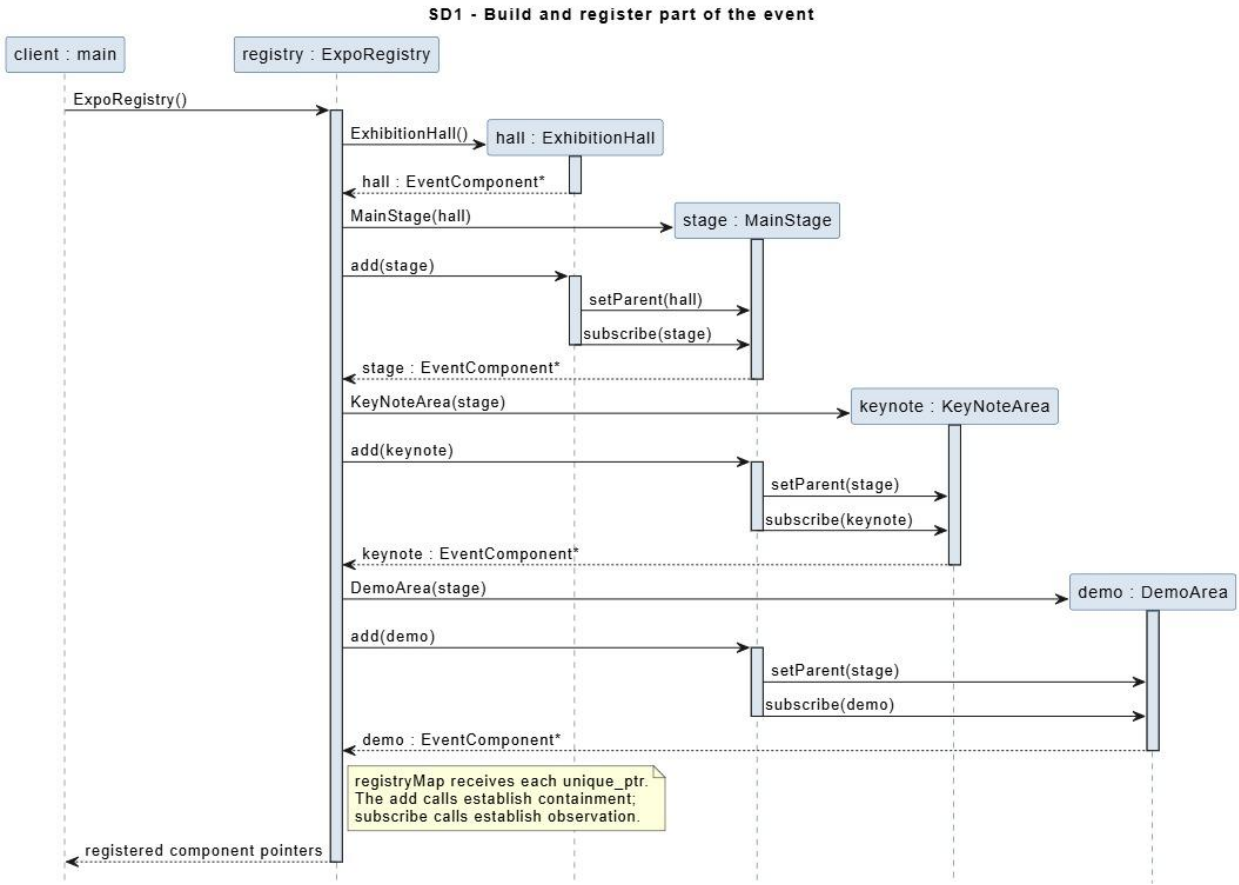
1. Original features Rotating keynote presenters. KeyNoteArea owns a BackgroundTimer that periodically invokes advancePresenter(). The timer is paused, resumed, or stopped by relevant notices, keeping time-management behaviour inside the keynote area rather than in a global controller.

2. Rotating technical demonstrations. DemoArea maintains an indexed demonstration schedule and technical specifications, with a separate BackgroundTimer that advances the active demonstration. Its safety and timer transitions remain local to the area.

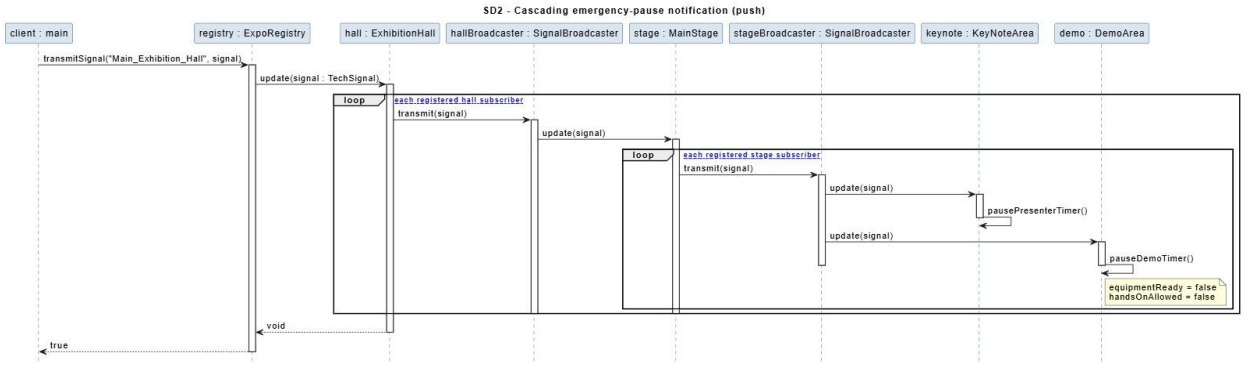
3. Technician rotation and escalation. MainStage keeps a rotating technician list and EventComponent::getStaff() delegates up the containment hierarchy. Units can include the on-duty technician in a safety or schedule-change response without owning staff-management logic.

4. Food-service inventory and orders. ByteSizedBites tracks seating, inventory, staff availability, active orders, and a daily special. These operational responsibilities are contained in the food-service leaf and do not make the registry or composite classes into god objects.

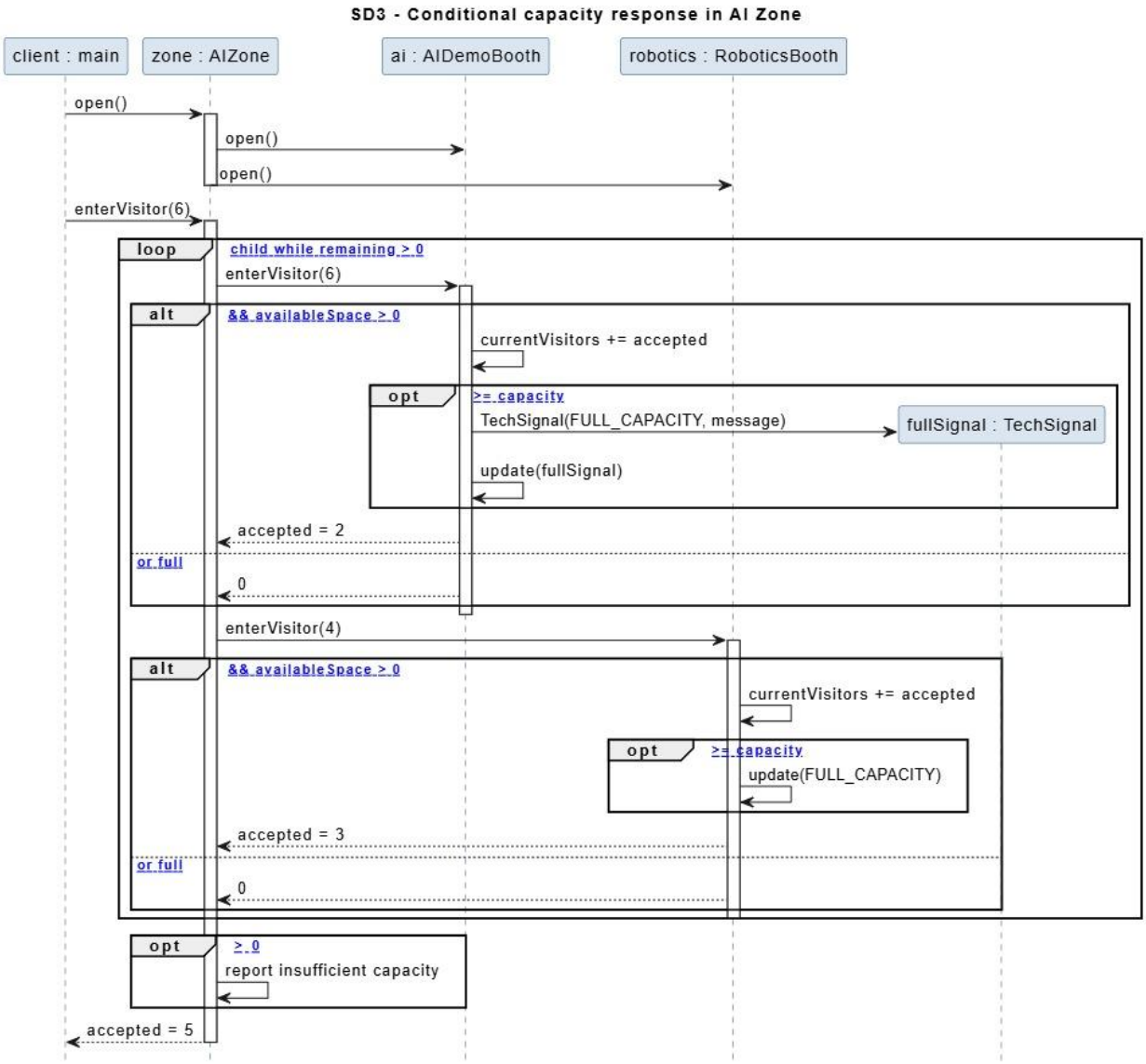
Task 7: SD1


SD1 - Build and register part of the event
client : main
registry : ExpoRegistry
ExpoRegistry()
ExhibitionHall()
hall : ExhibitionHall
hall : EventComponent*
MainStage(hall)
stage : MainStage
add(stage)
setParent(hall)
subscribe(stage)
stage : EventComponent*
KeyNoteArea(stage)
keynote : KeyNoteArea
add(keynote)
setParent(stage)
subscribe(keynote)
keynote : EventComponent*
DemoArea(stage)
demo : DemoArea
add(demo)
setParent(stage)
subscribe(demo)
demo : EventComponent*
registryMap receives each unique_ptr.
The add calls establish containment;
subscribe calls establish observation.
registered component pointers

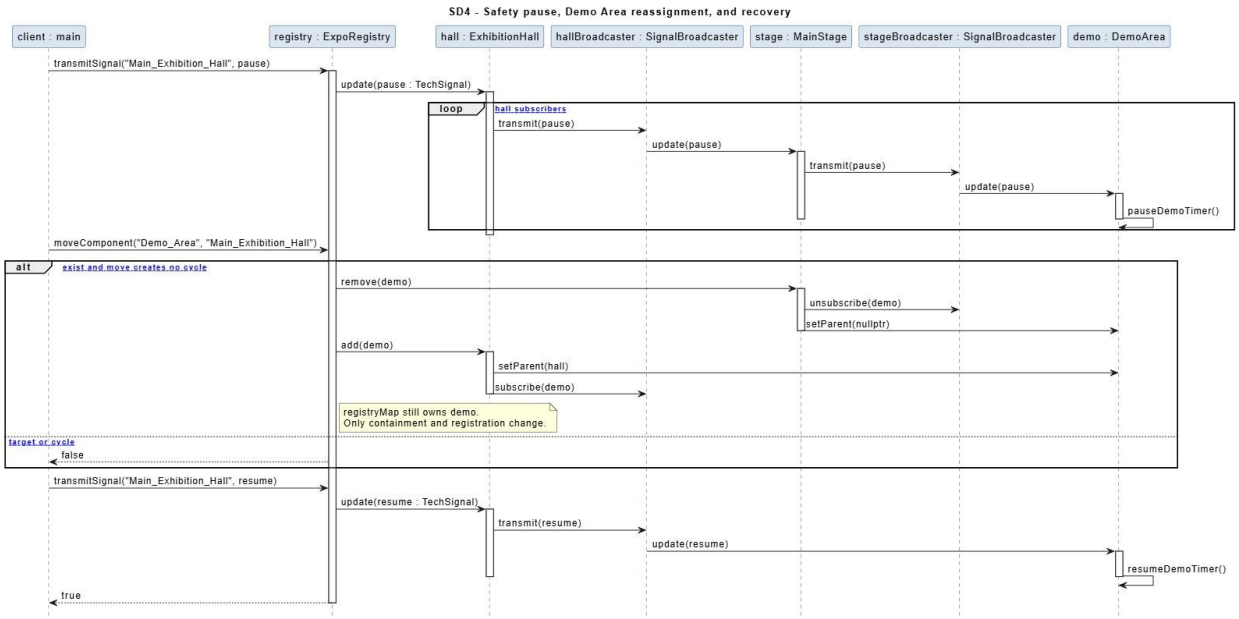
Task 8: SD2


SD2 - Cascading emergency-pause notification (push)
client : main
registry : ExpoRegistry
hall : ExhibitionHall
hallBroadcaster : SignalBroadcaster
stage : MainStage
stageBroadcaster : SignalBroadcaster
keynote : KeyNoteArea
demo : DemoArea
transmitSignal("Main_Exhibition_Hall", signal)
update(signal : TechSignal)
loop
each registered hall subscriber
transmit(signal)
update(signal)
loop
each registered stage subscriber
transmit(signal)
update(signal)
pausePresenterTimer()
update(signal)
pauseDemoTimer()
equipmentReady = false
handsOnAllowed = false
void
true

Task 9: SD3


SD3 - Conditional capacity response in AI Zone
client : main
zone : AIZone
ai : AIDemoBooth
robotics : RoboticsBooth
open()
open()
open()
enterVisitor(6)
loop
child while remaining > 0
enterVisitor(6)
alt
&& availableSpace > 0
currentVisitors += accepted
opt
>= capacity
TechSignal(FULL_CAPACITY, message)
fullSignal : TechSignal
update(fullSignal)
accepted = 2
or full
0
enterVisitor(4)
alt
&& availableSpace > 0
currentVisitors += accepted
opt
>= capacity
update(FULL_CAPACITY)
accepted = 3
or full
0
opt
> 0
report insufficient capacity
accepted = 5

Task 10: SD4


SD4 - Safety pause, Demo Area reassignment, and recovery
client : main
registry : ExpoRegistry
hall : ExhibitionHall
hallBroadcaster : SignalBroadcaster
stage : MainStage
stageBroadcaster : SignalBroadcaster
demo : DemoArea
transmitSignal("Main_Exhibition_Hall", pause)
update(pause : TechSignal)
loop
hall subscribers
transmit(pause)
update(pause)
transmit(pause)
update(pause)
pauseDemoTimer()
moveComponent("Demo_Area", "Main_Exhibition_Hall")
alt
exist and move creates no cycle
remove(demo)
unsubscribe(demo)
setParent(nullptr)
add(demo)
setParent(hall)
subscribe(demo)
registryMap still owns demo.
Only containment and registration change.
target or cycle
false
transmitSignal("Main_Exhibition_Hall", resume)
update(resume : TechSignal)
transmit(resume)
update(resume)
resumeDemoTimer()
true

Task 11: Doxygen screenshot

Task 12: GitHub workflow

We used a GitHub Flow strategy throughout the development of EventFlow. The main branch was kept as the shared, integrated version of the project, while each major feature implementation was developed in a separate feature branch. Once a feature was completed, we created a pull request and merged it back into the shared main branch.

Our general workflow was to branch from main, implement and test the feature in isolation, and only create a pull request once the feature was complete. After merging, we tested the integrated code on the main branch, particularly in areas that closely represented the scope of the feature that had been added. This helped us identify integration issues that may not have been apparent when working on an individual feature branch.

We also aimed to keep commits small and meaningful so that the development history accurately represented how the project evolved. In addition, we used GitHub Issues to keep track of outstanding tasks and organise the work that still needed to be completed.

Branches and pull requests were particularly useful for dividing the project into distinct areas of work and allowing members to develop features independently without interfering with the shared codebase. Pull requests also made it easier to integrate completed work and identify and resolve merge conflicts. We used the pull-request review functionality to

check that changes remained consistent with the overall architecture and to provide feedback to one another before merging.

GitHub Repository

Task 13: Team contribution statement

The practical was completed collaboratively by all 3 members of the team. Responsibilities were divided according to the different architectural design and documentation requirements of the practical while design decisions, integration and testing and final review were handled collaboratively. We had 2 major meetings where we discussed our visions and ideas for this project, one before implementing and one halfway through to clear misconceptions. We also made use of GitHub issues to track what still needs to be done.

Ethan

Oversaw designing the observer design pattern. Made design decisions such as making all the components of the hierarchy inherited from the observer and making composites use composition for making them a subject. I also designed and implemented the tech signal class which is a wrapper for a command/signal. I architected the ExpoRegistry however did not implement it. I was also responsible for doing the UML diagram, writing the design rationale, typing the event description and identifying the participants. Although I did not type all the Doxygen comments I created the Doxyfile and ensured it worked correctly.

Mogale

Created a background timer implementing the state design pattern. Implemented a template method to get the display details of a component. Assisted with debugging and implementing the ExpoRegistry and map builder. Assisted with implementing the leaves

and composites of the design pattern. I also created a technician class which makes use of operator overloading

Lesego

Participated in both team meetings, including the initial planning and development discussions.

Implemented three leaf classes and two composite classes, including the Exhibition Hall component.

Contributed to the testing and debugging phase, helping identify and resolve issues to ensure that all system components worked cohesively together.

Designed and completed the final sequence diagrams for the system, ensuring that they accurately represented the implemented functionality and interactions between components.